

ARCHITECTURE / PROTOCOL / VERIFICATION

Dual-Core CHI Cache Coherence Subsystem

Architecture, Protocol, and Verification Report

ARM AMBA CHI-B Protocol | RTL - Verilog / SystemVerilog | Vivado xsim

111 PASS | 0 FAIL

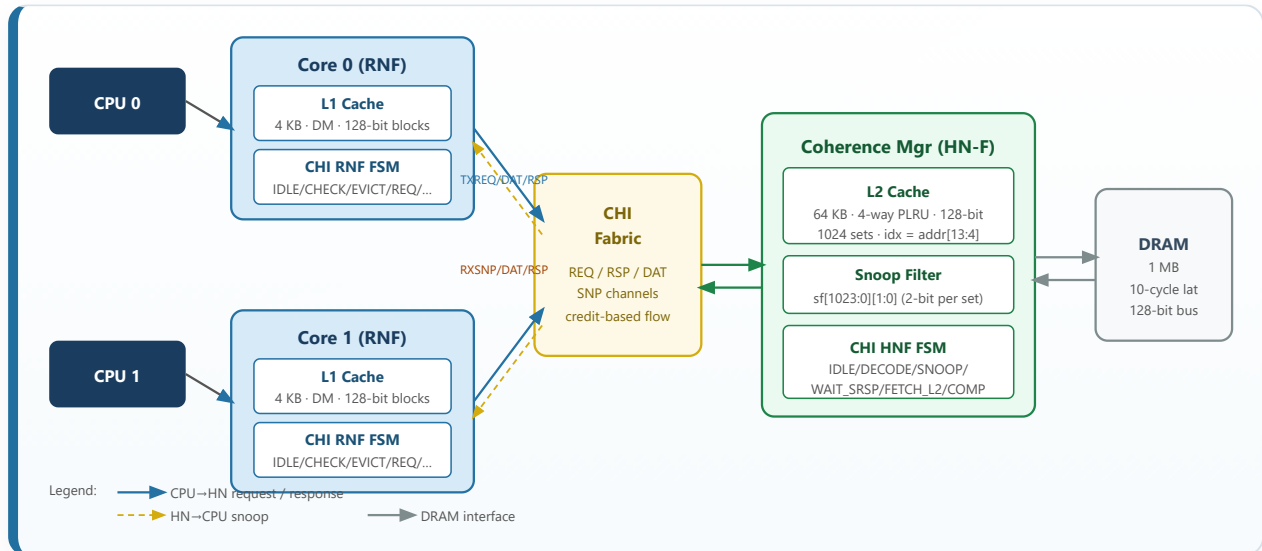
44 test groups

September 2026

DOCUMENT	Dual-Core CHI Cache Coherence Subsystem
SCOPE	Architecture, Protocol, and Verification Report
PROTOCOL	ARM AMBA CHI-B Protocol
RTL	Verilog / SystemVerilog
SIMULATION	Vivado xsim 111 PASS 0 FAIL across 44 test groups
DATE	September 2026

1 System Architecture

The design implements a dual-core shared-memory system with full CHI-B cache coherence. Each core contains its own private L1 instruction/data cache; a single shared L2 cache (with an integrated Home Node Function and Snoop Filter) arbitrates ownership of every cache line. An ARM-aligned CHI fabric routes all REQ, RSP, DAT, and SNP channel traffic between the Request Nodes (RNs) and the Home Node (HN-F).



1.1 Cache Geometry

Parameter	L1 Cache	L2 Cache	DRAM
Size	4 KB	64 KB	1 MB
Organization	256 sets, direct-mapped	1024 sets, 4-way PLRU	—
Block / line size	128 bits (16 bytes)	128 bits (16 bytes)	128 bits
Tag bits	addr[31:12] (20 b)	addr[31:14] (18 b)	—
Index bits	addr[11:4] (8 b)	addr[13:4] (10 b)	—
Word-offset	addr[3:2] (2 b)	addr[3:2] (2 b)	—
Conflict stride	0x1000	0x4000	—
Latency	1 cycle (hit)	2–3 cycles (tag + data)	10 cycles (fixed)
DRAM init pattern	mem[widx] = {0xDA00, widx[15:0]}, widx = byte_addr >> 2		

2 Module Descriptions

2.1 `cache_top.v` — Top-level Wrapper

Instantiates two `core` modules, one `chi_fabric`, one `coherence_manager`, and one `dram_model`. Exposes two 32-bit CPU interfaces and six 32-bit performance counter outputs (L1-0 hits/misses, L1-1 hits/misses, L2 hits/misses). All inter-module wiring is inside this file; no logic.

2.2 `core.v` — Request Node Function (RNF)

Combines a private `l1_cache` instance with a multi-state FSM that:

- Accepts 32-bit CPU reads/writes (with 4-bit byte-enable).
- Looks up the L1 cache and decides whether a CHI transaction is required.
- Issues TXREQ commands (ReadShared, ReadUnique, CleanUnique, WriteBackFull, Evict).
- Handles incoming RXDAT (CompData) and RXRSP (Comp) responses.
- Responds to incoming RXSNP snoops (SnpSharedFwd, SnpUnique) on the TXRSP / TXSNPDAT channels.
- Maintains per-line coherence state (I/SC/UC/UD/SD) inside the L1 array.
- Counts hits and misses via `perf_hits` / `perf_misses` output registers.

One instance (`u_core0` , `u_core1`) exists per CPU.

2.3 `l1_cache.v` — L1 Cache Array

Pure storage module. Contains:

Array	Depth	Width	Purpose
<code>data[255]</code>	256	128 bits	Cache line data
<code>tag[255]</code>	256	20 bits	Address tag
<code>cs[255]</code>	256	3 bits	Coherence state

Two read ports (port A for CPU requests, port B for snoop lookups) and one write port. Hit detection, coherence-state read, and data read are combinatorial on port A and B.

2.4 `coherence_manager.v` — Home Node Function (HN-F)

Central coherence point. Contains:

- **L2 cache** instance (`u_l2`) — 64 KB, 4-way PLRU.
- **Snoop Filter (SF)** — `sf[1023:0][1:0]` : bit 0 = core 0 has a copy, bit 1 = core 1 has a copy.
- **HNF FSM** — arbitrates all incoming REQ commands and orchestrates snoops, L2 reads/writes, and completions.

Only one request is in-flight at a time (no request pipelining). The SF is updated on every SF-altering snoop response: SnpUnique always clears the snoopee's bit; SnpSharedFwd clears only if the snoopee responded with `SnpRespI`.

2.5 `l2_cache.v` — L2 Cache Array

Four-way PLRU cache. Uses a standard 3-bit PLRU tree per set to select the way to replace. Accepts a single read or write request per cycle from `coherence_manager`; responds via `req_ack`. On a miss, issues a `dram_ren` to the DRAM model and waits for `dram_ack` before returning data.

2.6 `chi_fabric.v` — CHI Interconnect

Passive crossbar. Routes six CHI channels between the two cores and the HN-F using simple fixed-priority arbitration (core 0 wins over core 1 on shared channels). No buffering — all channels are zero-latency muxes. Credit signals (`lcrd` , `rdy` , `ack`) flow transparently.

2.7 `dram_model.v` — DRAM Behavioural Model

A 1 MB memory array (`mem[262143:0]` , 32 bits per word). Responds to 32-bit byte addresses; serves 128-bit blocks (four words). Fixed 10-cycle read latency via a countdown counter; writes are immediate. Initial contents: `mem[w] = {0xDA00, w[15:0]}`.

3 Cache Coherence States (MOESI/CHI Subset)

The design uses a five-state subset of the CHI-B protocol state machine. State is tracked per cache line inside the L1 `cs[255]` array (3 bits). The HN-F does not track L1 states directly — it uses the Snoop Filter to know which cores hold a copy.

State	Code	Meaning	Can Read?	Can Write?	Data Valid?
I	3'd0	Invalid — line not present in L1	—	—	—
SC	3'd1	Shared Clean — clean copy, may be shared with peer	✓	—	✓
UC	3'd2	Unique Clean — exclusive clean copy; transitional before write	✓	✓	✓
UD	3'd3	Unique Dirty — exclusive dirty copy; L2 stale; must write back on eviction	✓	✓	✓
SD	3'd4	Shared Dirty — master dirty copy; peer may hold SC; must write back or fwd on snoop	✓	—	✓

3.1 State Transition Summary

From	Event	To	CHI Opcode Issued
I	CPU read	SC	ReadShared → CompData (RESP_SC)
I	CPU write (miss)	UD	ReadUnique → CompData (RESP_UC) → local write
I	ReadUnique, peer was UD	UD	ReadUnique → CompData (RESP_UD, forwarded)
SC	CPU read hit	SC	— (L1 hit, no CHI)
SC	CPU write hit	UD	CleanUnique → Comp (RESP_UC) → local write
SC	Eviction	I	Evict → Comp
UC	CPU write (after CU grant)	UD	S_LOCAL_WR state applies write; no CHI
UD	CPU read/write hit	UD	— (L1 hit)
UD	Eviction	I	WriteBackFull → Comp
UD	SnpSharedFwd received	SD	SnpsRespDataFwd (dirty data → requester + HN)

From	Event	To	CHI Opcode Issued
UD	SnpUnique received	I	SnpRespData (dirty data → HN)
SD	CPU read hit	SD	— (L1 hit)
SD	CPU write hit	UD	CleanUnique → Comp → local write
SD	SnpUnique received	I	SnpRespData (dirty data → HN, HN writes L2)

4 CHI-B Protocol — Supported Opcodes

4.1 REQ Channel (RN → HN) — ReqOpcode[5:0]

Opcode	Hex	Name	L1 state required	Final L1 state	HN action
6'h01	0x01	ReadShared	I	SC	If peer in SF: SnpSharedFwd; else fetch L2/DRAM. Respond CompData RESP_SC.
6'h07	0x07	ReadUnique	I	UD	If peer in SF: SnpUnique. If peer was UD: forward dirty data as RESP_UD (no L2 write). Else fetch L2/DRAM, respond RESP_UC. Core applies CPU write in S_LOCAL_WR.
6'h0B	0x0B	CleanUnique	SC or SD	UD	Requester has data. If peer in SF: SnpUnique to invalidate. Responds Comp (permission only, no data). Core upgrades to UC then applies CPU write → UD in S_LOCAL_WR. If peer had dirty data (e.g. SD case): HN writes it to L2.
6'h0C	0x0C	MakeUnique	I	UD	Full cache-line store; requester overwrites entire line. Peer dirty data DISCARDED per spec. Responds Comp. <i>Not triggered at 32-bit CPU interface in this implementation</i> (would require full-line BE).
6'h0D	0x0D	Evict	SC or UC	I	Clean eviction; no data transfer. HN clears SF bit for requester, responds Comp. Next fill can skip snoop.
6'h1B	0x1B	WriteBackFull	UD or SD	I	Dirty eviction. Carries full 128-bit dirty line to HN. HN writes to L2 (or DRAM via L2), responds Comp.

4.2 SNP Channel (HN → RN) — SnpOpcode[4:0]

Opcode	Hex	Name	Required snooper action	Snooper final state
5'h01	0x01	SnpShared	Retain SC or downgrade from UD. Write dirty data to HN if dirty.	SC or I

Opcode	Hex	Name	Required snoopee action	Snoopee final state
5'h07	0x07	SnpUnique	Must invalidate. Write dirty data to HN if dirty.	I
5'h08	0x08	SnpClean	Clean dirty data, may stay SC.	SC or I
5'h09	0x09	SnpSharedFwd	If UD: downgrade to SD, forward dirty data directly to requester. If SC: respond clean (SC retained).	SD or SC

4.3 RSP Channel — RspOpcode[3:0]

Opcode	Hex	Name	Direction	Meaning
4'h1	0x1	SnpRespl	RN → HN	Snoop response: snoopee went Invalid (no data)
4'h2	0x2	SnpRespSC	RN → HN	Snoop response: snoopee retained Shared Clean (no data)
4'h4	0x4	Comp	HN → RN	Permission-only completion (CleanUnique, Evict, WriteBackFull)

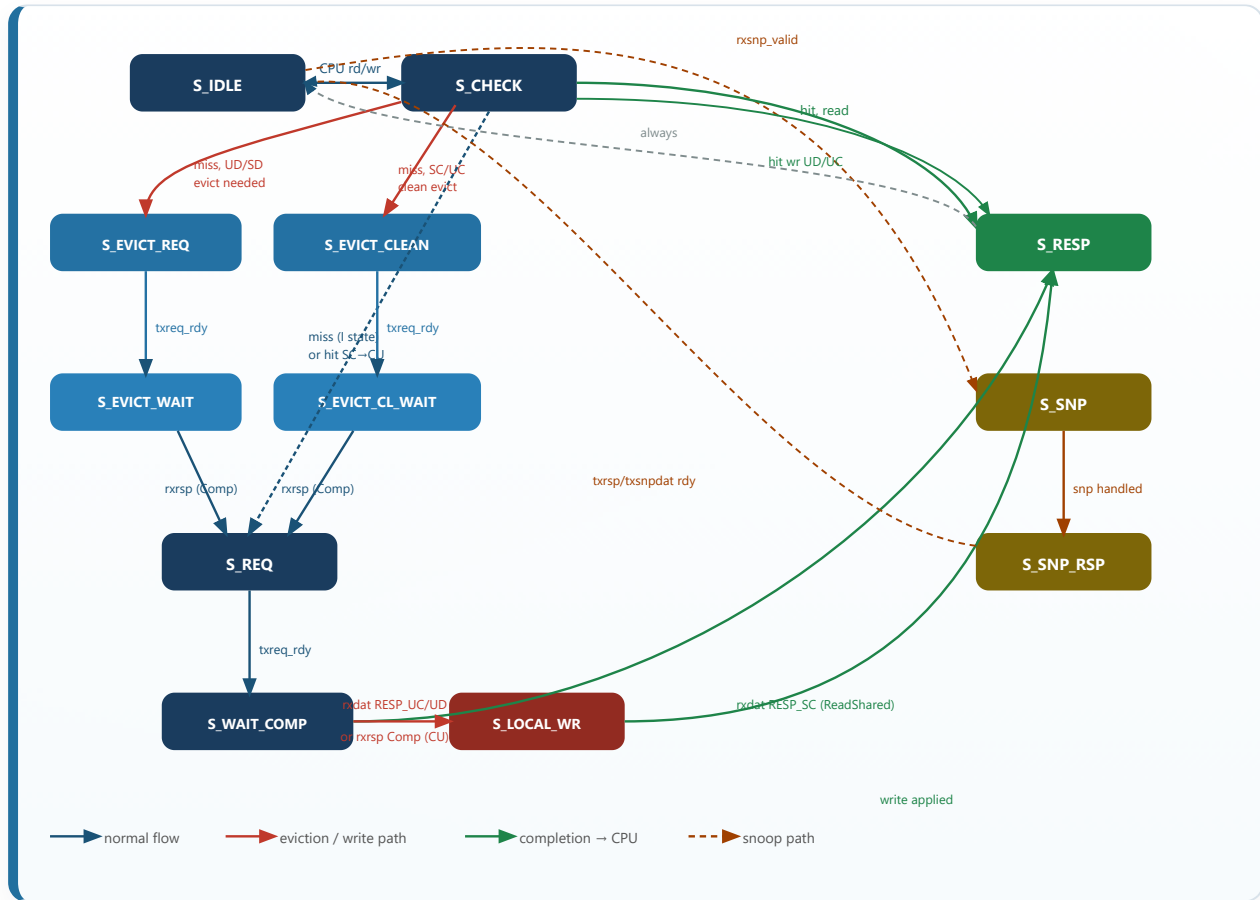
4.4 DAT Channel — DatOpcode[2:0] + Resp[2:0]

Opcode	Hex	Name	Direction	Resp[2:0]	Meaning
3'h1	0x1	SnpRespData	RN → HN	RESP_I / RESP_SC	Dirty snoop data to HN; snoopee went I or SC
3'h4	0x4	CompData	HN → RN	RESP_SC / RESP_UC / RESP_UD	Data + completion to requester; Resp encodes granted L1 state
3'h6	0x6	SnpRespDataFwd	RN → HN	RESP_SD	Dirty snoop data forwarded to requester AND to HN; snoopee stays SD

Resp[2:0]	Binary	State granted	Notes
RESP_I	3'b000	I	Line not retained by snoopee
RESP_UC	3'b010	UC	Requester gets Unique Clean (ReadUnique from clean L2)
RESP_UD	3'b011	UD	Requester gets Unique Dirty (forwarded from UD peer)
RESP_SC	3'b110	SC	Requester gets Shared Clean (ReadShared)
RESP_SD	3'b111	SD	Snoopee stays SD (SnpSharedFwd: holder keeps dirty master copy)

5 Core (RNF) FSM State Diagram

The core FSM has 12 states. CPU accesses always begin in **S_IDLE** → **S_CHECK**. The path through the FSM depends on the L1 hit/miss result and the coherence state of the line.

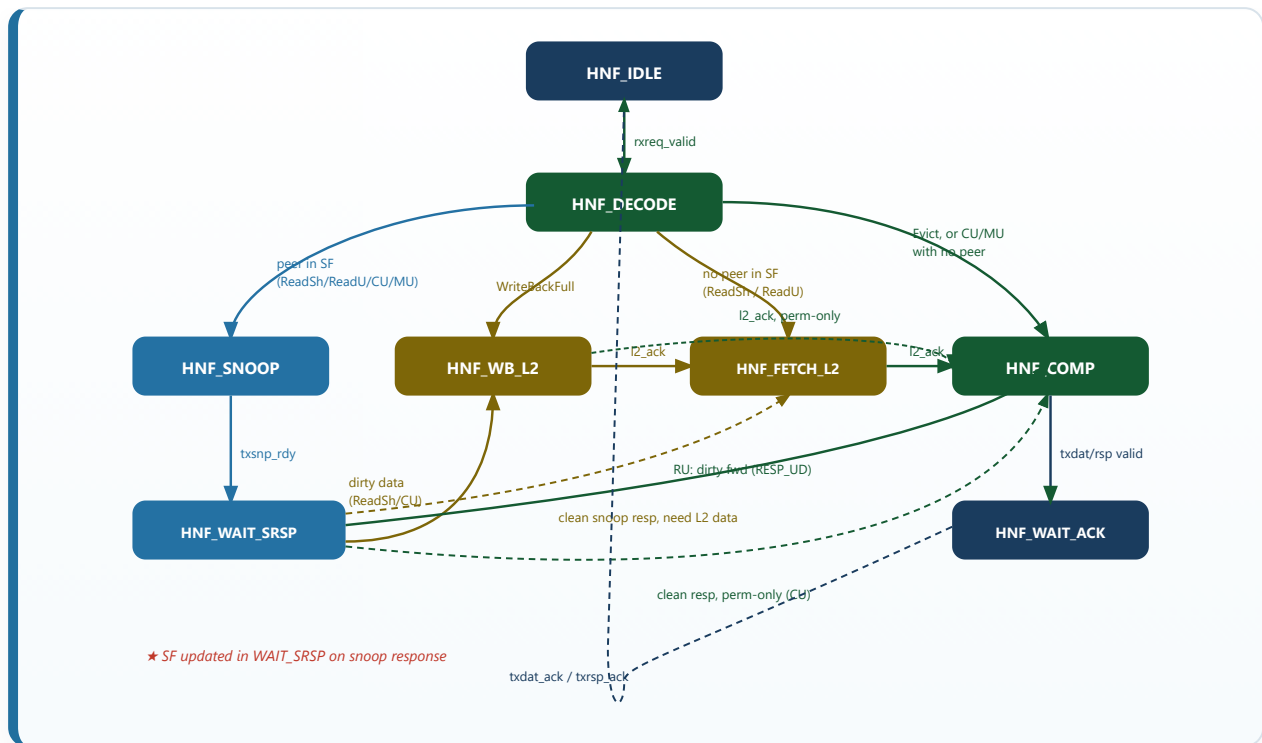


5.1 S_LOCAL_WR — Design Note

S_LOCAL_WR is a dedicated state introduced to correctly model CHI-B semantics where *getting permission* is separate from *applying the write*. After receiving CompData (RESP_UC or RESP_UD) for a ReadUnique, or a Comp for a CleanUnique, the line is first installed as **UC** in L1. **S_LOCAL_WR** then merges the pending CPU write data (byte-enabled) into the cached line and sets the state to **UD**. This ensures L2 never receives partially-merged data through the CHI channels.

6 Home Node (HN-F) FSM State Diagram

The HNF FSM is single-issue: only one request is processed at a time. On receipt of a RXREQ, the HN captures the opcode, source, address, and write-data, then decides the appropriate coherence action based on the SnooP Filter.



6.1 Key HNF State Descriptions

State	Description
HNF_IDLE	Waiting for a new REQ. Asserts rxreq_lcrd when req arrives; latches address, opcode, source ID, write data.
HNF_DECODE	One-cycle decision: examine opcode and Snoop Filter. Starts snoop, L2 fetch, L2 write, or goes directly to COMP.
HNF_SNOOP	Holding TXSNP valid until txsnp_rdy handshakes. Transitions to WAIT_SRSP.
HNF_WAIT_SRSP	Waits for snoop response on RSP channel (clean) or SNPDAT channel (dirty). Updates SF. Routes to WB_L2, FETCH_L2, or COMP based on what arrived and the original opcode.
HNF_WB_L2	Issues L2 write (dirty peer data or WBF payload). Waits for I2_ack. If requester needs data, transitions to FETCH_L2; otherwise COMP.
HNF_FETCH_L2	Issues L2 read. Waits for I2_ack; captures rdata. Transitions to COMP.
HNF_COMP	Sets up txdat_valid (CompData) or txrsp_valid (Comp). Updates SF to add requester. Transitions to WAIT_ACK.
HNF_WAIT_ACK	Waits for txdat_ack or txrsp_ack from fabric/core. Returns to IDLE.

6.2 ReadUnique Dirty Forwarding

When the peer holds a **UD** line and the requester issues ReadUnique, the HNF forwards the dirty data *directly* to the requester with `RESP_UD`. The L2 is **not** written at this point — the requester owns the authoritative copy. L2 is updated later when the requester evicts via WriteBackFull. This avoids an unnecessary L2 round-trip and matches CHI-B §4.3 dirty forward semantics.

7 Key Coherence Transaction Flows

7.1 Cold Read (I → SC)

```
CPU0 rd 0x100 → [L1-0] MISS (I)
CPU0 →HN REQ ReadShared addr=0x100
HN →L2 read (MISS→DRAM, 10-cycle)
HN →CPU0 CDAT CompData RESP_SC data=0xDA000040
CPU0 L1: line[0x10] = 0xDA000040, state=SC
```

7.2 Shared Read (SC peer) — SnpSharedFwd

```
CPU1 rd 0x100 → [L1-1] MISS (I)
CPU1 →HN REQ ReadShared addr=0x100 (CPU0 has SC; SF bit0=1)
HN →CPU0 SNP SnpSharedFwd addr=0x100
CPU0 →HN SRSP SnpRespSC (retained SC, no data)
HN →L2 read (HIT)
HN →CPU1 CDAT CompData RESP_SC data=...
Both CPU0, CPU1 in SC
```

7.3 Upgrade Write (SC → UD via CleanUnique)

```
CPU0 wr 0x100 → [L1-0] HIT (SC)
CPU0 →HN REQ CleanUnique addr=0x100 (peer CPU1 SC, SF bit1=1)
HN →CPU1 SNP SnpUnique addr=0x100
CPU1 →HN SRSP SnpRespI (invalidated, SF bit1 cleared)
HN →CPU0 COMP Comp
Core0 S_LOCAL_WR: merge CPU write into line, state→UD
```

7.4 Dirty Read (UD peer) — SnpSharedFwd with dirty forward

```
CPU1 rd 0x100 → [L1-1] MISS (I)
CPU1 →HN REQ ReadShared addr=0x100 (CPU0 has UD)
HN →CPU0 SNP SnpSharedFwd addr=0x100
CPU0 →HN SDAT SnpRespDataFwd RESP_SD data=0xDEADBEEF
CPU0 state → SD (keeps dirty master copy)
HN →L2 write (dirty data from snoop)
HN →CPU1 CDAT CompData RESP_SC data=0xDEADBEEF
CPU1 → SC, CPU0 → SD
```

7.5 ReadUnique — Dirty Peer Forwarding (RESP_UD)

```
CPU1 wr 0x100 → [L1-1] MISS (I) [CPU0 has UD]
CPU1 →HN REQ ReadUnique addr=0x100
HN →CPU0 SNP SnpUnique addr=0x100
CPU0 →HN SDAT SnpRespData RESP_I data=0xDEADBEEF (gave up dirty)
HN: req_opc == ReadUnique → forward dirty directly (NO L2 write)
HN →CPU1 CDAT CompData RESP_UD data=0xDEADBEEF
Core1 S_LOCAL_WR: merge CPU1 write → UD
CPU0 → I, CPU1 → UD (L2 stays stale; updated when CPU1 evicts via WBF)
```

7.6 Dirty Eviction (WriteBackFull)

```
CPU0 conflicts on addr 0x1100 [CPU0 has UD @ 0x100]
CPU0 →HN REQ WriteBackFull addr=0x100 wdata=dirty_line
HN →L2 write (dirty data → L2)
HN →CPU0 COMP Comp
CPU0 L1 line 0x10: state → I, tag cleared
CPU0 →HN REQ ReadUnique addr=0x1100 ...
```

8 Verification Test Suite

The testbench (`tb_cache_top.sv`) runs 44 test groups and 111 individual check points. All tests pass:

111 PASS | 0 FAIL. Simulation time: ~32,335 ns at 100 MHz (10 ns clock period).

8.1 Transaction Monitor

Each test is instrumented with always-block monitors that print one line per CHI handshake:

Tag	Trigger	Information printed
[L1-x]	Core enters S_CHECK	Address, current L1 state, HIT/MISS
[REQ]	hn_rxreq_valid & lcrd	Source core, opcode, address, wdata[31:0]
[SNP]	hn_txsnp_valid & rdy	Target core, snoop opcode, address
[SRSP]	hn_rxrsp_valid & lcrd	Source core, RSP opcode (clean response, no data)
[SDAT]	hn_rxsnpdat_valid & lcrd	Source core, DAT opcode, Resp (final state), data[31:0]
[CDAT]	hn_txdat_valid & ack	Target core, DAT opcode, Resp (granted state), data[31:0]
[COMP]	hn_txrsp_valid & ack	Target core, RSP opcode (permission grant)
[L2]	l2_hits or l2_misses change	HIT or MISS, cumulative count

8.2 Test Group Summary

Test	Description	CHI Opcodes Exercised	States Covered	Checks
1	Cold read CPU0	ReadShared	I→SC	1
2	CPU1 reads same line — SnpSharedFwd (SC peer)	ReadShared, SnpSharedFwd	SC+SC	1
3	CPU0 write on SC — CleanUnique, CPU1→I	CleanUnique, SnpUnique	SC→UD, SC→I	1
4	CPU1 reads UD — SnpSharedFwd, dirty fwd	ReadShared, SnpSharedFwd, SnpRespDataFwd	UD→SD, I→SC	2
5	CPU0 dirty eviction — WriteBackFull	CleanUnique, WriteBackFull	SD→I	—
6	CPU1 reads evicted address — L2 hit	ReadShared	I→SC (L2 hit)	1
7	CPU0 ReadUnique write, ICN SnpUnique to CPU1	CleanUnique, SnpUnique, ReadShared	SC→UD, I→SC	2
8	Write-allocate (write miss)	ReadUnique	I→UD	2
9	Byte-enable partial write	ReadUnique	I→UD (4-BE)	1
10	PLRU stress — 5 blocks → same L2 set	ReadUnique, WriteBackFull ×5	L2 PLRU evict	5

Test	Description	CHI Opcodes Exercised	States Covered	Checks
12	CPU1 write-allocate + hit coverage	ReadUnique, ReadShared	I→UD, UD→SC	2
13	False sharing — each CPU writes one word	CleanUnique, SnpUnique ×2	UD ping-pong	2
14	Three-way coherence chain	ReadShared, CleanUnique, SnpUnique, ReadShared	UD→SD→UD	2
15	Evict clears SF proactively	Evict	SC→I	2
16	CPU1 WBF eviction visible to CPU0	WriteBackFull, ReadShared	UD→I, L2 hit	1
21	Evict enables snoop-free fill	Evict, ReadShared	SC→I→SC	2
23	SnpSharedFwd: UD→SD, requester gets SC	ReadShared, SnpSharedFwd	UD→SD	2
24	MakeReadUnique: SD→UD via CleanUnique	CleanUnique, SnpUnique	SD→UD	2
25	Performance counter readback	—	hit/miss counts	2
26	Byte-enable — all individual byte combinations	ReadUnique	UD	7
27	All 4 words in a 128-bit cache line	ReadUnique, ReadShared, SnpSharedFwd	UD, SC	5
28	ReadUnique dirty forwarding — RESP_UD path	ReadUnique, SnpUnique (dirty fwd)	UD→I, I→UD	3
29	Round-robin write ownership	CleanUnique ×2, SnpUnique ×2	UD ping-pong	4
30	I→SC→UD sequence on one address	ReadShared, CleanUnique	I→SC→UD	3
31	SD CleanUnique — dirty data to HN via snoop path	CleanUnique, SnpUnique, ReadShared	SD→UD	4
32	Two independent addresses — no interference	ReadUnique ×2, ReadShared ×2	UD, SC (×2)	4
33	L1 capacity sweep — 8 non-conflicting sets	ReadUnique ×8, ReadShared ×8	UD ×8	8
34	Dirty eviction + CPU1 reads from L2	WriteBackFull, ReadShared, CleanUnique	UD→I, L2 hit	3
35	CPU1 write after CPU0 dirty eviction	ReadUnique, SnpSharedFwd	I→UD, UD→SD	3
36	Sparse address coverage — 10 unique L1 sets	ReadUnique ×10, ReadShared ×10	UD ×10	10
37	SC→UD writes, eviction, re-read from L2	CleanUnique, WBF, ReadShared	SC→UD→I→SC	4
38	CPU1 dirty eviction visible to CPU0	WBF, ReadShared, CleanUnique	UD→I, SC→UD	4
39	Adjacent cache lines — no cross-line interference	ReadUnique ×4	UD ×4	4
40	CPU1 cold reads from DRAM	ReadShared ×3	I→SC ×3	3
41	CPU1 CleanUnique from SC (symmetric upgrade)	CleanUnique, SnpUnique, SnpSharedFwd	SC→UD, SD→I	3
42	All 5 CHI opcodes on single address 0x0E20	RS, CU, WBF, RU, Evict	I→SC→UD→I→UD→I→SC	5
43	Two dirty eviction+refill cycles — same address	WBF ×2, ReadUnique, ReadShared	UD→I→UD→I	4
44	CPU1 SD→UD via CleanUnique (symmetric)	SnpSharedFwd, CleanUnique	SD→UD	3
Total				111

8.3 Coverage Matrix

Feature	Covered By	Result
I → SC (cold read)	Tests 1, 40	PASS
SC → UD (CleanUnique upgrade)	Tests 3, 7, 24, 29, 30, 41	PASS

Feature	Covered By	Result
I → UD (write-allocate / ReadUnique)	Tests 8, 9, 28, 35	PASS
UD → SD (SnpSharedFwd with dirty)	Tests 4, 23	PASS
SD → UD (CleanUnique from SD)	Tests 24, 31, 44	PASS
UD → I (WriteBackFull)	Tests 5, 16, 34, 37, 43	PASS
SC → I (Evict)	Tests 15, 21, 42	PASS
ReadUnique RESP_UD dirty forwarding	Test 28	PASS
SnpSharedFwd (UD → SD + SC forwarded)	Tests 2, 4, 7, 12, 14, 23	PASS
PLRU L2 replacement	Test 10	PASS
Byte-enable partial writes	Tests 9, 26	PASS
L1 conflict eviction (same idx, different tag)	Tests 5, 37, 43	PASS
Snoop Filter accuracy (snoop-free fill after Evict)	Test 21	PASS
Performance counters	Test 25	PASS
Multiple writes (UD hit, no protocol needed)	Tests 30, 37	PASS
All 5 CHI REQ opcodes on one address	Test 42	PASS
Symmetric CPU0/CPU1 paths	Tests 12, 38, 41, 44	PASS

9 Design Notes & Constraints

9.1 Single-Issue HNF

The coherence manager processes one REQ at a time. While a request is in-flight, `rxreq_lcrd` is de-asserted so no second request can enter HNF_DECODE. This serialises coherence operations but avoids deadlock from resource ordering.

9.2 No MakeUnique at 32-bit Interface

MakeUnique requires a *full cache-line store* (all 128 bits overwritten in one operation so the old data need not be fetched). Because the CPU interface is 32-bit with 4-bit byte-enable, a full-line store cannot be expressed in one transaction. All write misses therefore use ReadUnique to fetch the existing 96-bit context before merging.

9.3 S_LOCAL_WR Rationale

CleanUnique and ReadUnique return *permission* (UC state) not the modified data. The actual CPU write data is held in `req_core_wdata` and `req_core_be`. S_LOCAL_WR performs the byte-enabled merge into the now-UC line and promotes it to UD — all within the core, without any CHI traffic.

9.4 Snoop Filter Update Rules

SF bits are set for the requester in HNF_COMP. They are cleared for the snoopee when SnpUnique forces I (unconditionally), or when a SnpSharedFwd/SnpShared receives a `SnpRespI` (peer chose to evict rather than retain SC). The SF is initialised to all-zero on reset.

9.5 File List

File	Role	Lines (approx.)
rtl/cache_defines.v	L1/L2/DRAM geometry constants	27
rtl/chi_defines.v	CHI-B opcode & state constants	62
rtl/l1_cache.v	L1 SRAM array (256 × 128b + tag + cs)	~80
rtl/l2_cache.v	L2 4-way PLRU + DRAM FSM	~160
rtl/core.v	RNF FSM + L1 cache controller	~500
rtl/coherence_manager.v	HNF FSM + L2 + Snoop Filter	~420
rtl/chi_fabric.v	CHI crossbar (6 channels, 2 cores)	~200
rtl/cache_top.v	Top-level instantiation	~350
rtl/dram_model.v	Behavioural DRAM (10-cycle latency)	~70
tb/tb_cache_top.sv	Testbench — 44 test groups, 111 checks, CHI monitor	~820